Dr. Konrad Völkel Heinrich-Heine-Universität Düsseldorf
Wintersemester 2026/27

Einführung in Deep Learning
Blatt 4
(7 Punkte, abzugeben bis 12:00 am Mittwoch, dem 11.11.2026)

In diesem Blatt üben wir die ersten Schritte mit PyTorch.
Aufgabe 1 gibt einen Punkt, Aufgabe 2 gibt 6 Punkte.

Hinweise.
- *Wenn Sie die Trainingsroutine zu kompliziert finden, fangen Sie mit Test-Cases für das Training aus einer einzelnen Stichprobe an (z.B. mit den ersten 2 Features des Iris-Datensatzes) und bringen Sie die ans Laufen.*
- *Der Loss sollte im Training sinken, nicht notwendig strikt monoton, aber im Mittel. Die Kurve muss oben links anfangen, runtergehen und unten bleiben; nach einiger Zeit ändert sich der Loss nur noch wenig. Plotten Sie* `losses` *gegen* `epoch`*, bei sehr vielen Epochen lieber nur die ersten.*
- *Zum Debuggen ausdrücklich* alle *Daten ins Training stecken und keinen Train-Test-Split machen: Overfitting ist erst ein Problem, wenn überhaupt etwas gelernt wurde. Der Split kommt danach.*
- *Wenn der Loss bei sehr sauber getrennten Daten nie aufhört zu sinken und die Gewichte immer größer werden, ist das erwartetes Verhalten, siehe die Warnung im Kapitel "Loss-Funktionen und Maximum Likelihood". Wenn der Loss* `nan` *wird, siehe die Warnung zur Numerik von* e^x *und* $\ln$ *im Kapitel "Logistische Regression":*
  `https://eidl-skript-131e30.pages.cs.uni-duesseldorf.de/loss-funktionen-maximum-likelihood`
  `https://eidl-skript-131e30.pages.cs.uni-duesseldorf.de/logistische-regression/`

Implementierung mit PyTorch

Aufgabe 1. *Installieren Sie, sofern nicht schon geschehen, PyTorch in einer Version größer als 2, z.B. 2.5 oder 2.15. Dabei sollten Sie darauf achten, direkt die korrekte Plattform auszuwählen (Linux/Mac/Windows, CUDA/ROCm/CPU, Conda/Pip). Möglicherweise müssen Sie eine aktuellere Python-Version installieren (für PyTorch 2.15 mind. Python 3.11).*

Auf einem Ubuntu 22.04-System mit Python 3.10 oder Debian 12 (stable, Bookworm) mit Python 3.11 genügt etwa der Befehl

```
python3 -m pip install torch torchvision torchaudio
```

Testen Sie Ihre Installation auf der Python-Kommandozeile mit

```
import torch
torch.cuda.is_available()
torch.backends.mps.is_available()
torch.backends.cpu.get_cpu_capability()
```

Wenn Sie NVIDIA-Hardware nutzen möchten, machen Sie sich mit dem Output von `nvidia-smi` *und* `nvcc --version` *vertraut, insbesondere der CUDA Version (z.B. 13.2). Wenn Sie AMD-Hardware nutzen möchten, machen Sie sich mit dem Output von* `rocminfo | grep gfx` *vertraut. Wenn Sie Apple Silicon (M1,M2,...) nutzen möchten, machen Sie sich mit dem Begriff "Metal Performance Shader" (MPS) vertraut.*

Dokumentieren Sie in Ihrer Abgabe den Output der Methoden und was Sie erwartet hatten, was der Output sein sollte bzw. ob sich der Output mit Ihren Erwartungen deckt. Schreiben Sie

Einführung in Deep Learning
Blatt 4
(7 Punkte, abzugeben bis 12:00 am Mittwoch, dem 11.11.2026)

---

dazu, ob Sie auf Ihrer CPU rechnen wollen, oder über eine Tensor-Beschleunigung (wie z.B. eine NVIDIA GPU) verfügen.

Aufgabe 2. *Implementieren Sie ein einzelnes Perzeptron (nicht ein verallgemeinertes, sondern das 'gute alte' wie auf dem voringen Übungsblatt) mit PyTorch anstelle von Numpy (also z.B. mit* `torch.where` *statt* `np.where` *und mit Torch Tensoren statt Numpy Ndarrays). Nutzen Sie Scikit-Learn mit* `sklearn.datasets.make_moons` *um einen Datensatz zu erstellen, den Sie in Train- und Test-Datensatz splitten. Lassen Sie ihr Perzeptron auf dem Train-Datensatz lernen (dazu müssen Sie die Daten von* `make_moons` *in einen Torch Tensor umwandeln) und visualisieren Sie die resultierende Klassifikation im Test-Datensatz mit Matplotlib (dazu müssen Sie den Torch-Tensor in einen Numpy Array umwandeln).*

Wenn Sie möchten (Bonusaufgabe), können Sie Hardware-Beschleunigung jenseits der CPU verwenden - aber ihr Programm sollte auch auf der CPU laufen (also auch auf einem Rechner ohne Hardware-Beschleunigung).

Ein verallgemeinertes Perzeptron bzw. eine ganze Schicht davon bietet `torch.nn.Linear`*, aber das wollen wir hier ausrücklich verbieten. Ebenfalls wollen wir in dieser Aufgabe noch auf autograd, die automatische Differenzierungs-Engine (autodiff), und damit automatischen Gradientenabstieg, verzichten und stattdessen direkt die Perzeptron-Update-Regel aus den Vorlesungsfolien verwenden.*

Einführung in Deep Learning
Blatt 4
(7 Punkte, abzugeben bis 12:00 am Mittwoch, dem 11.11.2026)

Strukturieren Sie Ihren Code so, dass Sie eine Methode wie diese implementieren:
`def perceptron_loss(label, true_label):`
die zurückgibt, wie viele Einträge in dem eindimensionalen Tensor `label` von dem Eintrag in `true_label` mit gleichem Index abweichen.

Implementieren Sie auch die Methode `heaviside(x)` (die Heaviside step function mit Wert 0 bei $x < 0$ und 1 bei $x > 0$; wir wählen den Wert 0 bei $x = 0$) um die Perzeptron-Inferenz umzusetzen.

Strukturieren Sie Ihren Code so, dass Sie Methoden `perceptron_infer(x, params)` für die Inferenz und `optimizer_step(x, y, y_pred, params)` für das Training implementieren, die jeweils als Input mehrere Datenpunkte zugleich (einen Mini-Batch oder alles auf einmal) als Torch Tensor annehmen und auch zurückgeben. Die Trainingsroutine könnte dann so strukturiert sein, wenn der Input ein Shape-`(N,d)`-Tensor `x` und ein Shape-`(N,)`-Tensor `y` sind, also N Datenpunkte $x_1, x_2, \ldots, x_N \in \mathbb{R}^d$ mit Labels $y_1, \ldots, y_N \in \{0, 1\}$:

```
params = initial_weight(x)
for n in range(EPOCHS):
    y_pred = perceptron_infer(x, params)
    loss = perceptron_loss(y_pred, y)
    print(f"Loss at {loss} in epoch {n}")
    params = optimizer_step(x, y, y_pred, params)
```

Um mehrere Datenpunkte auf einmal in `optimizer_step(x, y, y_pred, params)` zu verarbeiten, müssen Sie sich entscheiden, ob Sie durch den Mini-Batch mit einer for-Schleife iterieren und dabei jeweils die Perzeptron-Update-Regel für ein einzelnes Update durchführen, oder ob Sie sich selbst eine Variante ausdenken, die mehrere Einzelupdates auf einmal durchführt. Achtung: Die Arbeit mit Mini-Batches kann etwas 'bürokratisch' werden. Im ersten Anlauf sollten Sie ohne Batching arbeiten und somit in jeder Epoche einzeln durch die Datenpunkte iterieren. Das Minibatching ist die Extra-Herausforderung für besonders ambitionierte.